

B.TECH 1ST YEAR · COMPUTER SCIENCE & ENGINEERING

Data Structures

Unit IV — Queues & Deques

A comprehensive, exam-oriented study module covering all aspects of Queue data structures, implementations, applications, and Double-Ended Queues with detailed algorithms and diagrams.

SUBJECT CODE

CS-101

UNIT

IV of V

TOPICS

14 Sections

01	Introduction to Queues	3
02	Properties of Queues	4
03	Queue Operations – Enqueue, Dequeue, Peek	5
04	Queue Implementation using Arrays	6
05	Queue Implementation using Linked Lists	8
06	Circular Queue	10
07	Applications of Queues	12
08	BFS using Queue	13
09	CPU Scheduling using Queue	15
10	Introduction to Deques	16
11	Operations on Deques	17
12	Applications of Deques	18
13	Questions & Answers (2-mark & 5-mark)	19
14	Quick Revision Sheet	22
—	Textbook & Reference Books	23

 **Exam Note:** This module covers the complete syllabus of Unit IV as per B.Tech 1st Year curriculum. Every topic includes definition, algorithm, diagram, complexity analysis, and exam-oriented Q&A. Study the diagrams carefully — they appear frequently in university examinations.

SECTION 01

Introduction to Queues

DEFINITION

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle — the element inserted first is the first one to be removed. A queue is open at both ends: one end is used exclusively for insertion (*rear/back*) and the other for deletion (*front*).

The concept of a queue is analogous to a real-world queue (line) of people waiting at a ticket counter — whoever joins first gets served first. In computing, queues are fundamental to scheduling, buffering, and traversal algorithms.

Historical Context & Analogy

The queue abstract data type was formally studied by Alan Turing and John von Neumann during early computer architecture work. Today, queues underpin operating systems, networking stacks, and AI search algorithms.

Queue vs Stack

Property	Queue	Stack
Principle	FIFO (First In, First Out)	LIFO (Last In, First Out)
Insertion end	Rear	Top
Deletion end	Front	Top
Real-world analogy	Bank queue / printer spooler	Stack of plates / call stack
Primary use	Scheduling, BFS, buffering	Recursion, DFS, expression eval

FIG 1.1 – QUEUE REPRESENTATION (FIFO PRINCIPLE)



Elements enter from the REAR and leave from the FRONT — demonstrating FIFO ordering.

Key Terminology

- Front:** The end from which elements are removed (dequeued).
- Rear:** The end at which elements are added (enqueued).
- Overflow:** Attempting to enqueue into a full queue.
- Underflow:** Attempting to dequeue from an empty queue.

- **Capacity:** Maximum number of elements a queue can hold (for array implementation).

SECTION 02

Properties of Queues

Understanding the fundamental properties of queues is essential for choosing the right data structure in algorithm design. The following properties characterize all queue implementations:

Fundamental Properties

1. **FIFO Ordering:** The first element inserted is always the first to be removed. The relative order of elements is preserved.
2. **Two-Ended Access:** Insertion is restricted to one end (rear) and deletion to the other (front).
3. **Sequential Storage:** In array-based queues, elements occupy contiguous memory locations.
4. **Bounded / Unbounded:** Array implementations have a fixed maximum size; linked-list implementations grow dynamically.
5. **Single-Ended Traversal:** Unlike arrays, you cannot directly access the *i*-th element — traversal starts from the front.
6. **No Random Access:** Elements in the middle cannot be directly inserted or deleted.

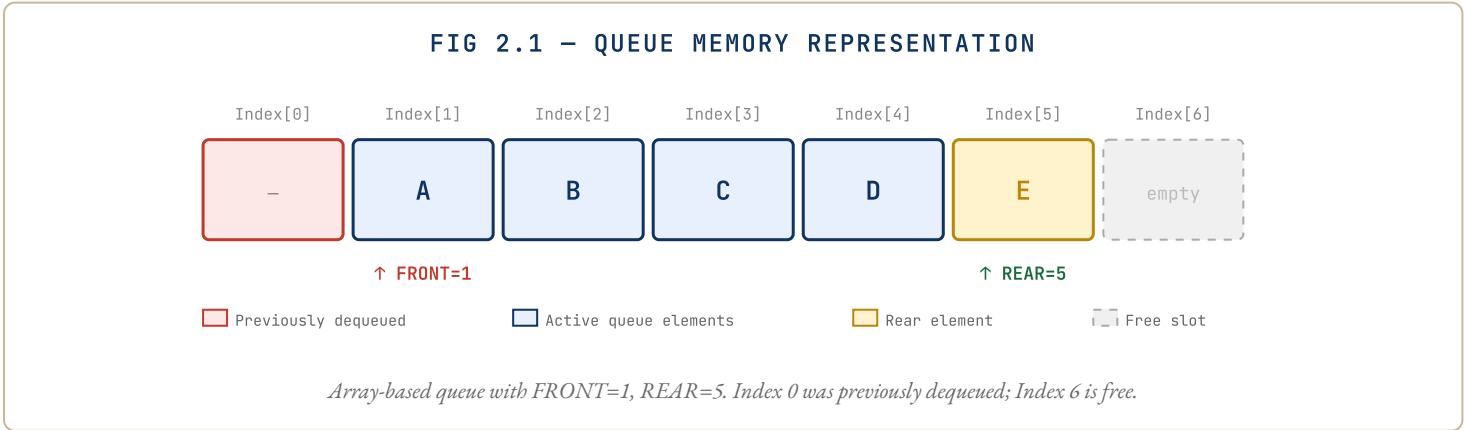
Mathematical Properties

Queue Properties – Formula Reference

Size of Queue = rear - front + 1 (for linear array queue)
Empty Condition = front > rear OR front == -1
Full Condition = rear == MAX - 1 (for array of size MAX)
Circular Empty = front == rear
Circular Full = (rear + 1) % MAX == front
Number of elements in circular queue = (rear - front + MAX) % MAX

Structural Representation

In memory, a queue implemented with an array of size N is represented as follows:



Queue as an Abstract Data Type (ADT)

As an ADT, a queue is defined by its operations and the invariants they maintain, independent of any specific implementation. The following invariants must always hold:

- If the queue is empty, `front = -1` (or `front > rear`).
- Elements are always added at rear and removed at front.
- The ordering of elements in the queue at any time reflects their insertion order.

Three primary operations define queue behavior: **Enqueue**, **Dequeue**, and **Peek/Front**.

3.1 Enqueue Operation

DEFINITION

Enqueue inserts an element at the *rear* of the queue. It first checks for overflow (queue full), then increments the rear pointer, and places the new element at the updated rear position.

Algorithm — Enqueue

```
ALGORITHM Enqueue(Q, element)
// Q is the queue structure, element is the value to insert
INPUT  : Queue Q, element to be inserted
OUTPUT : Updated queue with element at rear

Step 1: IF rear == MAX - 1 THEN
        PRINT "Queue Overflow - Cannot insert"
        RETURN
    END IF

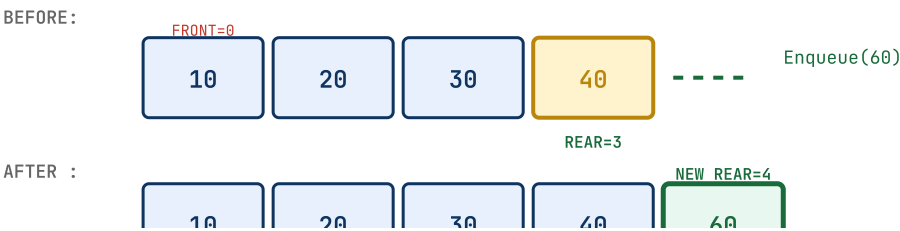
Step 2: IF front == -1 THEN    // Queue was empty
        front ← 0
    END IF

Step 3: rear ← rear + 1        // Move rear pointer forward

Step 4: Q[rear] ← element      // Insert element at rear

Step 5: PRINT "Inserted:", element
Step 6: RETURN
```

FIG 3.1 – ENQUEUE OPERATION (INSERTING 60 AT REAR)



Element 60 is inserted at the rear. REAR pointer advances from 3 to 4.

3.2 Dequeue Operation

DEFINITION

Dequeue removes and returns the element at the *front* of the queue. It checks for underflow (queue empty), retrieves the front element, and advances the front pointer.

Algorithm — Dequeue

```
ALGORITHM Dequeue(Q)
INPUT  : Queue Q
OUTPUT : Removed element (or error if empty)

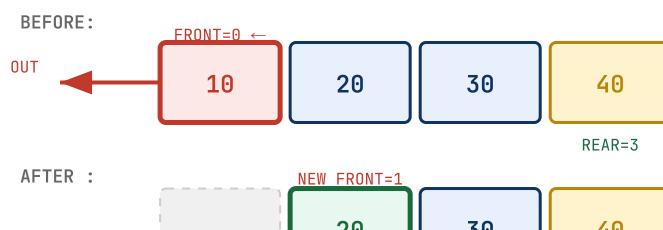
Step 1: IF front == -1 THEN
    PRINT "Queue Underflow - Queue is Empty"
    RETURN -1
END IF

Step 2: element ← Q[front]    // Save front element

Step 3: IF front == rear THEN // Queue becomes empty after removal
    front ← -1
    rear  ← -1
ELSE
    front ← front + 1 // Advance front pointer
END IF

Step 4: RETURN element
```

FIG 3.2 – DEQUEUE OPERATION (REMOVING FROM FRONT)



Element 10 is removed. FRONT pointer advances from 0 to 1. The freed slot is no longer accessible.

3.3 Peek / Front Operation

```
ALGORITHM Peek(Q)
Step 1: IF front == -1 THEN PRINT "Queue Empty" : RETURN
Step 2: RETURN Q[front]    // View without removing
```

🕒 Time Complexity of Queue Operations

Enqueue : $O(1)$ – Insert at rear, just increment pointer
Dequeue : $O(1)$ – Remove at front, just increment pointer
Peek : $O(1)$ – Read $Q[\text{front}]$ directly
isEmpty : $O(1)$ – Check $\text{front} == -1$
isFull : $O(1)$ – Check $\text{rear} == \text{MAX} - 1$

Traversal: $O(n)$ – Visit each element once

Space : $O(n)$ – n = number of elements stored

CONCEPT

The simplest queue implementation uses a **one-dimensional array** with two integer variables, `front` and `rear`, to track the head and tail of the queue. Elements occupy contiguous memory positions between indices `front` and `rear` inclusive.

Structure Declaration (C Language)

```
#define MAX 10                // Maximum queue capacity

typedef struct {
    int data[MAX];            // Array to store elements
    int front;                // Points to front element (-1 if empty)
    int rear;                 // Points to rear element (-1 if empty)
} Queue;

// Initialize
void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

// Check empty
int isEmpty(Queue *q) {
    return (q->front == -1);
}

// Check full
int isFull(Queue *q) {
    return (q->rear == MAX - 1);
}

// Enqueue
void enqueue(Queue *q, int val) {
    if (isFull(q)) {
        printf("Queue Overflow!\n");
        return;
    }
    if (isEmpty(q)) q->front = 0;
    q->data[++q->rear] = val;
    printf("Enqueued: %d\n", val);
}

// Dequeue
int dequeue(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int val = q->data[q->front];
    if (q->front == q->rear) // Last element
        q->front = q->rear = -1;
```

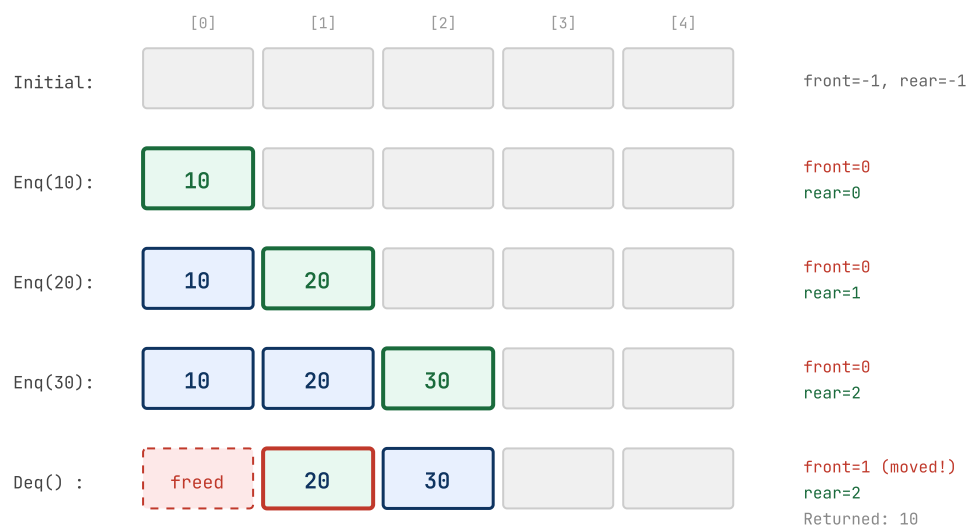
```

else
    q->front++;
return val;
}

// Display
void display(Queue *q) {
    if (isEmpty(q)) { printf("Queue is empty\n"); return; }
    printf("Queue [front → rear]: ");
    for (int i = q->front; i ≤ q->rear; i++)
        printf("%d ", q->data[i]);
    printf("\n");
}

```

FIG 4.1 – QUEUE USING ARRAY (STEP-BY-STEP STATE)



State progression of array-based queue through enqueue and dequeue operations.

✓ Advantages

- Simple to implement
- $O(1)$ enqueue and dequeue
- Memory-efficient for fixed-size queues
- Cache-friendly (contiguous memory)

✗ Disadvantages

- Fixed capacity — cannot grow dynamically
- Memory wastage after many dequeues
- Front slots cannot be reused (solved by circular queue)
- Overflow even with free slots at front

CONCEPT

A queue implemented using a **singly linked list** uses nodes dynamically allocated in heap memory. Each node stores a data value and a pointer to the next node. The **front** pointer points to the head (dequeue end) and the **rear** pointer to the tail (enqueue end).

Node Structure (C Language)

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *front;    // Head of list (dequeue here)
    Node *rear;     // Tail of list (enqueue here)
} LLQueue;

void initQueue(LLQueue *q) {
    q->front = q->rear = NULL;
}

int isEmpty(LLQueue *q) {
    return (q->front == NULL);
}

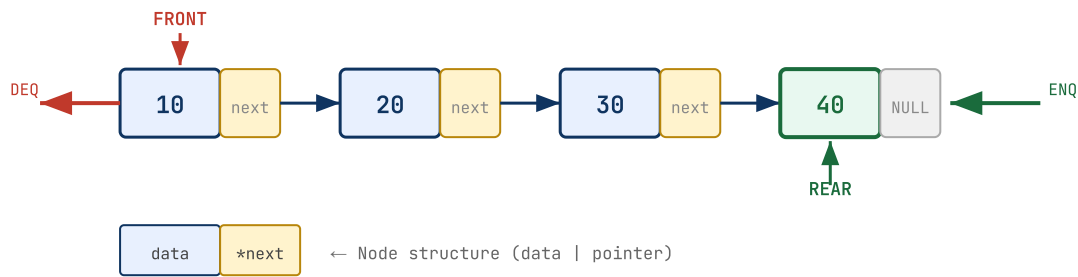
// Enqueue - O(1) because rear pointer maintained
void enqueue(LLQueue *q, int val) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = NULL;
    if (isEmpty(q)) {
        q->front = q->rear = newNode;
    } else {
        q->rear->next = newNode; // Link new node at end
        q->rear = newNode;      // Move rear pointer
    }
    printf("Enqueued: %d\n", val);
}

// Dequeue - O(1)
int dequeue(LLQueue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow!\n");
        return -1;
    }
    Node *temp = q->front;
    int val = temp->data;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL; // Queue now empty
    free(temp);
}
```

```
return val;
```

```
}
```

FIG 5.1 – QUEUE USING LINKED LIST



Linked list queue: FRONT points to head (dequeue side); REAR points to tail (enqueue side). NULL terminates the list.

Comparison: Array vs Linked List Queue

Criterion	Array Queue	Linked List Queue
Memory allocation	Static (compile time)	Dynamic (runtime, heap)
Size flexibility	Fixed (MAX constant)	Grows/shrinks dynamically
Enqueue time	$O(1)$	$O(1)$
Dequeue time	$O(1)$	$O(1)$
Memory overhead	None (just array)	Extra pointer per node
Cache performance	Better (contiguous)	Worse (scattered heap)
Overflow possible?	Yes (fixed size)	No (until heap exhausted)
Implementation complexity	Simple	Moderate

✓ Advantages of LL Queue

- No overflow (dynamic size)
- Memory used only as needed
- Easy insertion/deletion at $O(1)$

✗ Disadvantages of LL Queue

- Extra memory for pointers
- malloc/free overhead
- No random access

DEFINITION

A **Circular Queue** (also called a Ring Buffer) is a linear data structure in which the array positions are treated as if they form a circle. When the rear reaches the last index, it wraps around to index 0 if the front has moved forward, thus reusing freed memory slots.

Circular queues solve the major problem of linear array queues: *the queue appears full even when there are free slots at the front*. By using modular arithmetic ($\% \text{MAX}$), the rear and front pointers wrap around.

Circular Queue Conditions

Circular Queue – Key Formulas

Empty Condition : $\text{front} = -1$

Full Condition : $(\text{rear} + 1) \% \text{MAX} = \text{front}$

Enqueue rear : $\text{rear} = (\text{rear} + 1) \% \text{MAX}$

Dequeue front : $\text{front} = (\text{front} + 1) \% \text{MAX}$

Count of elements: $(\text{rear} - \text{front} + \text{MAX}) \% \text{MAX}$

[when $\text{front} \neq -1$]

Algorithm — Circular Queue Enqueue

```
ALGORITHM CircularEnqueue(Q, element)
  Step 1: IF  $(\text{rear} + 1) \% \text{MAX} = \text{front}$  THEN
    PRINT "Circular Queue Full"
    RETURN
  END IF
  Step 2: IF  $\text{front} = -1$  THEN    // First insertion
     $\text{front} \leftarrow 0$ 
     $\text{rear} \leftarrow 0$ 
  ELSE
     $\text{rear} \leftarrow (\text{rear} + 1) \% \text{MAX}$     // Circular increment
  END IF
  Step 3:  $Q[\text{rear}] \leftarrow \text{element}$ 
  Step 4: RETURN
```

Algorithm — Circular Queue Dequeue

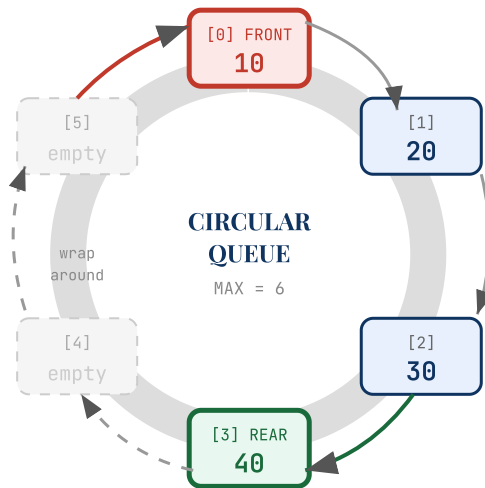
```
ALGORITHM CircularDequeue(Q)
  Step 1: IF  $\text{front} = -1$  THEN
    PRINT "Circular Queue Empty"
    RETURN -1
  END IF
  Step 2:  $\text{element} \leftarrow Q[\text{front}]$ 
```

```

Step 3: IF front == rear THEN // Only one element was present
    front ← -1
    rear ← -1
ELSE
    front ← (front + 1) % MAX // Circular increment
END IF
Step 4: RETURN element

```

FIG 6.1 – CIRCULAR QUEUE REPRESENTATION (MAX = 6)



Circular Queue with MAX=6. FRONT at [0], REAR at [3]. Slots [4] and [5] are free. When REAR wraps to [4] or [5], freed slots are reused.

Circular Queue State Table

Operation	front	rear	Queue State (MAX=5)	Note
Initial	-1	-1	[_ _ _ _ _]	Empty
Enqueue(10)	0	0	[10 _ _ _ _]	First element
Enqueue(20)	0	1	[10 20 _ _ _]	—
Enqueue(30)	0	2	[10 20 30 _ _]	—
Dequeue()	1	2	[_ 20 30 _ _]	Returns 10
Enqueue(40)	1	3	[_ 20 30 40 _]	—
Enqueue(50)	1	4	[_ 20 30 40 50]	—
Enqueue(60)	1	0	[60 20 30 40 50]	Wraparound! rear=(4+1)%5=0

SECTION 07

Applications of Queues

Queues appear in nearly every area of computer science, from operating systems to networking and artificial intelligence. Their FIFO ordering makes them indispensable for fair, ordered processing.

Major Application Areas

1. CPU Scheduling (FCFS — First Come First Served)

Processes waiting for CPU time are stored in a ready queue. The process at the front is given CPU time; new processes join at the rear. This ensures fairness and prevents starvation in simple scheduling policies.

2. Breadth-First Search (BFS)

BFS uses a queue to explore a graph level by level. The source vertex is enqueued; as each vertex is dequeued, its unvisited neighbors are enqueued. This ensures all nodes at distance d are visited before nodes at distance $d+1$.

3. I/O Buffers and Spooling

Print spoolers queue print jobs; keyboard input buffers store keystrokes in order. Network packets are buffered in queues before transmission to handle speed mismatches between sender and receiver.

4. Simulation of Real-World Queues

Bank queues, supermarket checkouts, call center waiting lines, and airport boarding queues are all modeled using queue data structures for discrete-event simulation.

5. Asynchronous Data Transfer

Message queues (like POSIX MQ, RabbitMQ) decouple producers and consumers; producer enqueues messages, consumer dequeues them asynchronously without direct coupling.

6. Tree and Graph Level-Order Traversal

Printing a binary tree level by level (BFS traversal of trees) uses a queue to process nodes level by level.

7. Multiprogramming / Job Scheduling

In batch processing systems, jobs submitted to a mainframe are queued and executed in submission order.

8. Network Traffic Management

Routers maintain packet queues; Quality of Service (QoS) systems use priority queues to manage traffic.

Application Domain	Queue Type Used	FIFO Reason
CPU Scheduling (FCFS)	Simple Queue	Process fairness
BFS Graph Traversal	Simple Queue	Level-by-level expansion
Printer Spooler	Simple Queue	Job ordering

Keyboard Buffer	Circular Queue	Wrap-around efficiency
OS Process Scheduling	Priority Queue	Priority + fairness
Message Queue Systems	Simple/Priority	Async decoupling
Traffic Simulation	Simple Queue	Arrival-order service

DEFINITION

Breadth-First Search (BFS) is a graph traversal algorithm that explores all vertices of a graph at the current depth level before moving to vertices at the next depth level. A queue is used to keep track of the vertices discovered but not yet fully explored.

BFS Algorithm

```
ALGORITHM BFS(Graph G, source vertex s)
  INPUT  : Graph G = (V, E), starting vertex s
  OUTPUT : All vertices visited in BFS order

  Step 1: Create an empty Queue Q
  Step 2: Create a visited[] boolean array, initialize all to FALSE
  Step 3: Mark visited[s] ← TRUE
  Step 4: Enqueue(Q, s)

  Step 5: WHILE Q is not empty DO
      u ← Dequeue(Q)           // Get next vertex
      PRINT u                  // Process vertex u

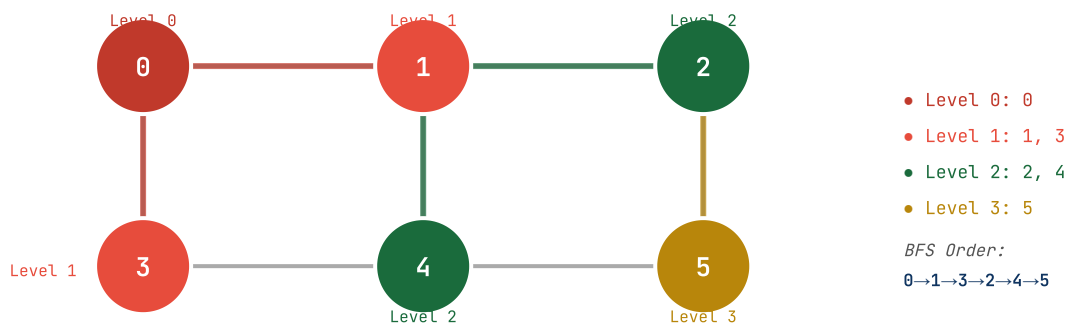
      FOR EACH neighbor v of u in adjacency list DO
          IF visited[v] = FALSE THEN
              visited[v] ← TRUE
              Enqueue(Q, v)    // Schedule v for exploration
          END IF
      END FOR
  END WHILE

  Step 6: STOP
```

BFS — Worked Example

Consider the following undirected graph with 6 vertices (0–5):

FIG 8.1 – BFS TRAVERSAL DIAGRAM (STARTING FROM VERTEX 0)



BFS traversal from vertex 0. Color indicates BFS level. BFS tree edges are highlighted.

BFS Step-by-Step Table

Step	Dequeued	Neighbors	Enqueued (new)	Queue State	Visited
Start	—	—	0	[0]	{0}
1	0	1, 3	1, 3	[1, 3]	{0,1,3}
2	1	0, 2, 4	2, 4	[3, 2, 4]	{0,1,2,3,4}
3	3	0, 4	— (already visited)	[2, 4]	{0,1,2,3,4}
4	2	1, 5	5	[4, 5]	{0,1,2,3,4,5}
5	4	1, 3	— (all visited)	[5]	{0,1,2,3,4,5}
6	5	2	— (all visited)	[] empty	{0,1,2,3,4,5}

🕒 BFS Time & Space Complexity

Time Complexity : $O(V + E)$ – V vertices, E edges (adjacency list)
 $O(V^2)$ – if using adjacency matrix
Space Complexity : $O(V)$ – queue stores at most V vertices
BFS finds shortest path in unweighted graphs (by edge count)

Applications of BFS

- **Shortest path** in unweighted graphs (social networks, maps)
- **Web crawling** — exploring web pages level by level from a seed URL
- **Garbage collection** — tri-color marking in JVM/Go GC
- **Peer-to-peer networks** — finding closest peers
- **Broadcasting** in computer networks

DEFINITION

CPU Scheduling is the operating system mechanism of deciding which process in the ready queue gets CPU time next. In FCFS (First Come First Served) scheduling, processes are served in the exact order they arrive — a direct application of the queue ADT.

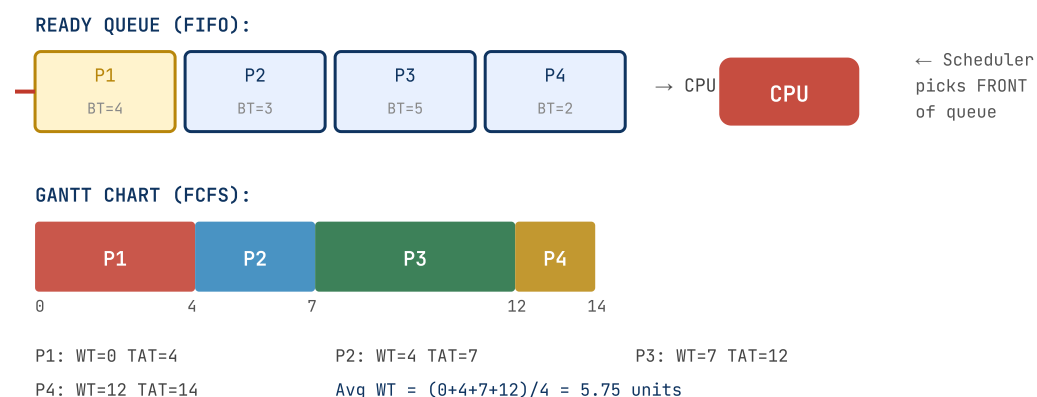
FCFS Scheduling — Algorithm

```

ALGORITHM FCFS_Scheduling(processes[], arrival[], burst[])
  Step 1: Enqueue all processes in arrival order into ReadyQueue
  Step 2: current_time ← 0
  Step 3: WHILE ReadyQueue not empty DO
    p ← Dequeue(ReadyQueue)      // Get next process
    IF current_time < arrival[p] THEN
      current_time ← arrival[p] // CPU idle wait
    END IF
    start_time[p]      ← current_time
    finish_time[p]     ← current_time + burst[p]
    waiting_time[p]    ← start_time[p] - arrival[p]
    turnaround_time[p] ← finish_time[p] - arrival[p]
    current_time       ← finish_time[p]
  END WHILE
  Step 4: COMPUTE average waiting time, average turnaround time
  Step 5: STOP

```

FIG 9.1 – CPU READY QUEUE & GANTT CHART



FCFS CPU Scheduling. Processes are served in arrival order (queue principle). Gantt chart shows CPU allocation timeline. WT = Waiting Time, TAT = Turnaround Time.

Round-Robin Scheduling (uses Circular Queue)

Round-Robin scheduling gives each process a fixed time quantum. After the quantum expires, the process is moved to the rear of the circular queue, and the next process is dequeued from the front. This uses a circular queue internally for efficient rear-to-front

wrapping.

Scheduling Formulas

Waiting Time (WT) = Completion Time - Arrival Time - Burst Time

Turnaround Time (TAT) = Completion Time - Arrival Time

Response Time = First CPU allocation time - Arrival Time

CPU Utilization = (Total burst time / Total time) × 100%

Throughput = No. of processes completed / Unit time

DEFINITION

A **Deque** (Double-Ended Queue, pronounced "deck") is a generalization of the queue data structure in which elements can be inserted and removed from *both ends* — the front and the rear. It combines the features of both stacks and queues.

The word "deque" was coined by E.J. Dijkstra. Unlike a queue (FIFO only) or a stack (LIFO only), a deque supports all four operations: push and pop at both front and rear.

Types of Deques

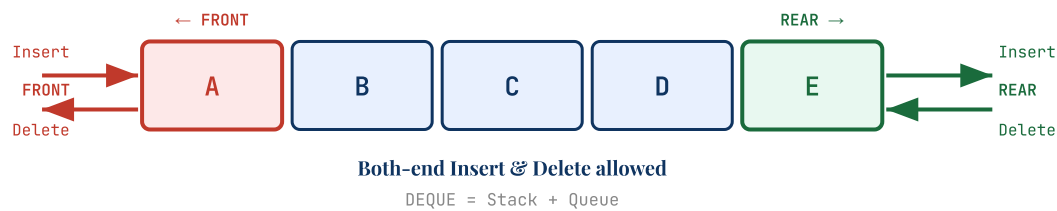
Input-Restricted Deque

Insertion is allowed only at the **rear** (one end), but deletion is allowed at both front and rear. Acts like a queue with the option to delete from the back.

Output-Restricted Deque

Deletion is allowed only at the **front** (one end), but insertion is allowed at both front and rear. Acts like a queue with the option to push to the front.

FIG 10.1 – DEQUE REPRESENTATION



A Deque supports insertion and deletion at BOTH the front and rear ends.

Deque vs Queue vs Stack

Operation	Stack	Queue	Deque
Insert at front	X	X	✓
Delete at front	X	✓	✓
Insert at rear	✓ (push)	✓	✓
Delete at rear	✓ (pop)	X	✓
Access principle	LIFO	FIFO	Both

SECTION 11 Operations on Deques

A deque supports five primary operations. All can be implemented in $O(1)$ time using either a doubly linked list or a circular array.

11.1 InsertFront — Insert at Front

```
ALGORITHM InsertFront(D, element)
  Step 1: IF isFull(D) THEN PRINT "Overflow" : RETURN
  Step 2: IF isEmpty(D) THEN
    front ← rear ← 0
  ELSE
    front ← (front - 1 + MAX) % MAX // Circular wrap
  END IF
  Step 3: D[front] ← element
  Step 4: RETURN
```

11.2 InsertRear — Insert at Rear

```
ALGORITHM InsertRear(D, element)
  Step 1: IF isFull(D) THEN PRINT "Overflow" : RETURN
  Step 2: IF isEmpty(D) THEN
    front ← rear ← 0
  ELSE
    rear ← (rear + 1) % MAX
  END IF
  Step 3: D[rear] ← element
  Step 4: RETURN
```

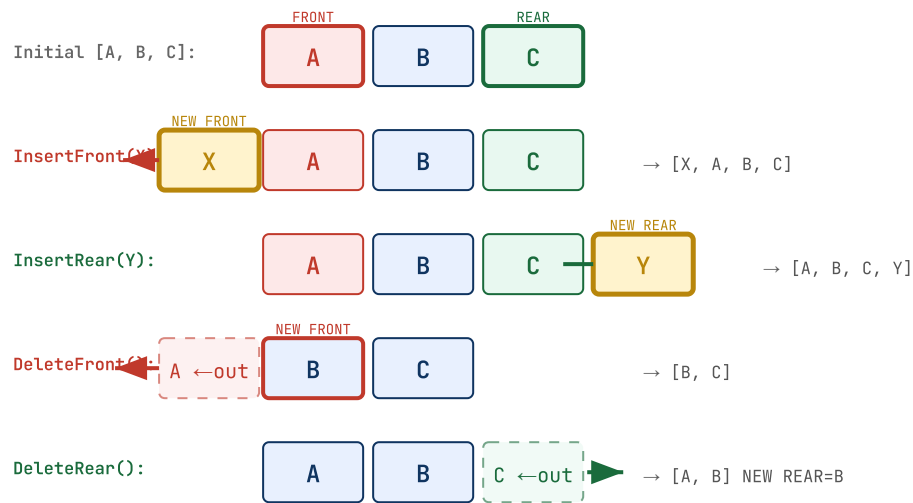
11.3 DeleteFront — Delete from Front

```
ALGORITHM DeleteFront(D)
  Step 1: IF isEmpty(D) THEN PRINT "Underflow" : RETURN -1
  Step 2: element ← D[front]
  Step 3: IF front == rear THEN front ← rear ← -1 // Now empty
    ELSE front ← (front + 1) % MAX
  Step 4: RETURN element
```

11.4 DeleteRear — Delete from Rear

```
ALGORITHM DeleteRear(D)
  Step 1: IF isEmpty(D) THEN PRINT "Underflow" : RETURN -1
  Step 2: element ← D[rear]
  Step 3: IF front == rear THEN front ← rear ← -1
    ELSE rear ← (rear - 1 + MAX) % MAX
  Step 4: RETURN element
```

FIG 11.1 – DEQUE INSERTION & DELETION OPERATIONS



Four deque operations shown step-by-step. X/Y are new elements; old elements removed are crossed out.

⌚ Deque Operations – Time Complexity

InsertFront : $O(1)$

InsertRear : $O(1)$

DeleteFront : $O(1)$

DeleteRear : $O(1)$

PeekFront : $O(1)$

PeekRear : $O(1)$

Search : $O(n)$ – linear scan needed

Space : $O(n)$ – n = number of stored elements

SECTION 12

Applications of Deques

Dequeues are more versatile than both stacks and queues due to their bidirectional nature. They appear in several high-performance algorithms.

1. Sliding Window Maximum / Minimum

In the sliding window maximum problem (given an array and a window size k , find the maximum in each window of size k), a deque stores indices of useful elements. Elements outside the window are deleted from the front; elements smaller than the new element are deleted from the rear. This gives $O(n)$ time vs $O(nk)$ with brute force.

2. Palindrome Checking

A string is pushed character-by-character into a deque. Then, characters are simultaneously removed from the front and rear and compared. If all pairs match, the string is a palindrome. This runs in $O(n)$ time.

3. Steal Scheduling (Work-Stealing Deque)

In parallel computing, worker threads maintain deques of tasks. A thread pops tasks from its own front; idle threads can *steal* tasks from the rear of another thread's deque. This is used in Java's ForkJoin framework and Go's goroutine scheduler.

4. Implementing Both Stack and Queue with One Deque

A single deque can simulate a stack (use only InsertRear/DeleteRear) or a queue (use InsertRear/DeleteFront), making it the most general sequential ADT.

5. Undo/Redo in Text Editors

Editor history can be maintained in a deque: undo pops from one end, redo from the other. This allows bidirectional history navigation efficiently.

6. A-Steal Algorithm in Multiprocessor Systems

The A-Steal work-stealing algorithm maintains a deque per processor. New tasks are added to the front; the local processor dequeues from the front; stealing processors take from the rear.

Deque Operations Comparison Table

Operation	Stack equivalent	Queue equivalent	Deque
Insert at front	N/A	N/A	InsertFront — $O(1)$
Insert at rear	Push — $O(1)$	Enqueue — $O(1)$	InsertRear — $O(1)$
Delete at front	N/A	Dequeue — $O(1)$	DeleteFront — $O(1)$

Delete at rear	Pop — $O(1)$	N/A	DeleteRear — $O(1)$
Peek front	N/A	Peek — $O(1)$	PeekFront — $O(1)$
Peek rear	Top — $O(1)$	N/A	PeekRear — $O(1)$

 **Exam Tip:** Questions about deques often ask you to "simulate a stack using deque" or "simulate a queue using deque." Remember: for stack behavior, use only rear operations (InsertRear + DeleteRear). For queue behavior, use InsertRear + DeleteFront.

Part A — 2-Mark Questions

Q1. Define a Queue. State its basic principle.

2 Marks

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle. It is an ordered collection in which the addition of new elements takes place at one end, called the **rear**, and the removal of existing elements occurs at the other end, called the **front**.

The FIFO principle means that the element inserted first will be the first one to be removed. This is analogous to a queue of people at a ticket counter — the person who arrives first is served first. Queues are used in scheduling, buffering, BFS traversal, and asynchronous message passing systems.

Q2. What is meant by Queue Overflow and Queue Underflow?

2 Marks

Queue Overflow occurs when an attempt is made to enqueue (insert) an element into a queue that is already full. In an array-based queue, this happens when $\text{rear} = \text{MAX} - 1$. It indicates that the queue has reached its maximum storage capacity and no more elements can be added without data loss.

Queue Underflow occurs when an attempt is made to dequeue (remove) an element from a queue that is already empty. This happens when $\text{front} = -1$ or $\text{front} > \text{rear}$. Both conditions must be checked before performing enqueue and dequeue operations respectively to ensure program correctness and prevent runtime errors.

Q3. What is a Circular Queue? Why is it preferred over a linear queue?

2 Marks

A **Circular Queue** is a queue implementation in which the array positions are treated logically as a circle. When the rear pointer reaches the last index (MAX-1), it wraps around to index 0, provided there is free space at the front of the array. The condition for a full circular queue is $(\text{rear} + 1) \% \text{MAX} = \text{front}$.

It is preferred over a linear queue because in a linear queue, once the rear reaches the end of the array, no more elements can be inserted even if elements have been dequeued from the front, leaving empty spaces. The circular queue solves this problem by reusing those freed slots through modular arithmetic, resulting in significantly better memory utilization.

Q4. What is a Deque? What are its two types?

2 Marks

A **Deque (Double-Ended Queue)** is a generalization of the queue data structure that allows insertion and deletion at both ends — the front and the rear. It combines the capabilities of both a stack and a queue in a single structure. All four operations (InsertFront, InsertRear, DeleteFront, DeleteRear) run in $O(1)$ time.

The two types of deques are: (1) **Input-Restricted Deque** — insertion is allowed only at the rear, but deletion is permitted from both front and rear; (2) **Output-Restricted Deque** — deletion is allowed only at the front, but insertion is permitted at both front and rear.

Q5. State the time complexity of all basic queue operations.

2 Marks

The time complexities of all basic queue operations are as follows: **Enqueue** — $O(1)$, since we simply increment the rear pointer and assign the value; **Dequeue** — $O(1)$, since we only read and increment the front pointer; **Peek/Front** — $O(1)$, just return `Q[front]` without modifying the queue.

isEmpty check — $O(1)$, compare front and rear; **isFull** check — $O(1)$, compare rear to `MAX-1`; **Traversal** — $O(n)$, must visit each element exactly once; **Space complexity** — $O(n)$ for n stored elements. These constant-time operations make queues highly efficient for scheduling and buffering applications.

Part B — 5-Mark Questions

Q6. Explain Queue implementation using Array with diagram, algorithm, and example.

5 Marks

Array-based Queue: A queue can be implemented using a one-dimensional array of fixed size `MAX` with two integer variables — `front` (initialized to -1) and `rear` (initialized to -1). The `front` variable always points to the first element of the queue, and `rear` points to the last element.

Enqueue Algorithm: First check if `rear == MAX - 1` (overflow). If the queue is empty (`front == -1`), set `front = 0`. Then increment rear and store the element: `Q[++rear] = element`.

Dequeue Algorithm: First check if `front == -1` (underflow). Retrieve the element: `val = Q[front]`. If `front == rear`, reset both to -1. Otherwise, increment front: `front++`. Return `val`.

Example: With `MAX=5`, perform `Enqueue(10)`, `Enqueue(20)`, `Enqueue(30)`, `Dequeue()`. After `Enqueue(10)`: `front=0`, `rear=0`, `queue=[10]`. After `Enqueue(20)`: `front=0`, `rear=1`, `queue=[10,20]`. After `Enqueue(30)`: `front=0`, `rear=2`, `queue=[10,20,30]`. After `Dequeue()`: returns 10, `front=1`, `rear=2`, `queue=[_,20,30]`.

Advantages: Simple implementation, $O(1)$ all operations. **Disadvantage:** Fixed size; memory wastage at front slots after multiple dequeues (solved by circular queue).

Time Complexity: `Enqueue` = $O(1)$, `Dequeue` = $O(1)$, `Peek` = $O(1)$, `Traversal` = $O(n)$. **Space Complexity:** $O(\text{MAX})$ always, regardless of actual queue size.

Q7. Explain Circular Queue with example, conditions, and step-by-step working.**5 Marks**

Circular Queue Definition: A circular queue treats the array as a ring. When rear reaches MAX-1, it wraps to index 0. This reuses slots freed by previous dequeue operations, eliminating the "false overflow" problem of linear queues.

Key Conditions: Empty: $\text{front} = -1$. Full: $(\text{rear} + 1) \% \text{MAX} = \text{front}$. Circular increment of rear: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$. Circular increment of front: $\text{front} = (\text{front} + 1) \% \text{MAX}$.

Enqueue Algorithm: (1) Check if $(\text{rear}+1)\% \text{MAX} = \text{front} \rightarrow$ overflow. (2) If first element, set $\text{front}=0, \text{rear}=0$. Else, $\text{rear} = (\text{rear}+1)\% \text{MAX}$. (3) Set $Q[\text{rear}] = \text{element}$.

Dequeue Algorithm: (1) Check if $\text{front} = -1 \rightarrow$ underflow. (2) Save $\text{val} = Q[\text{front}]$. (3) If $\text{front} = \text{rear}$, reset both to -1. Else, $\text{front} = (\text{front}+1)\% \text{MAX}$. (4) Return val.

Example with MAX=4: Enqueue(A): $f=0, r=0$. Enqueue(B): $f=0, r=1$. Enqueue(C): $f=0, r=2$. Dequeue(): returns A, $f=1, r=2$. Enqueue(D): $f=1, r=3$. Enqueue(E): $r=(3+1)\%4=0, f=1, r=0$. Queue now holds [E,B,C,D] wrapping around. Check full: $(0+1)\%4=1==\text{front}$, so full!

Advantage over Linear Queue: Efficient use of memory; no false overflow. Used as keyboard ring buffer, audio streaming buffer, and OS I/O buffers.

Q8. Explain BFS (Breadth-First Search) using Queue with algorithm, diagram, and step table.**5 Marks**

BFS Definition: Breadth-First Search is a graph traversal algorithm that visits all vertices of a graph level by level, starting from a given source vertex. A queue is used to maintain the order of vertices to be visited. BFS guarantees the shortest path (in terms of number of edges) from source to any reachable vertex in an unweighted graph.

BFS Algorithm: (1) Create empty queue Q and visited[] array. (2) Mark source as visited; enqueue source. (3) While Q is not empty: dequeue vertex u; print u; for each neighbor v of u — if not visited: mark visited[v]=true; enqueue v. (4) Repeat until queue is empty.

Worked Example (Graph: 0-1, 0-3, 1-2, 1-4, 2-5, 3-4): Start at vertex 0. Enqueue 0. Step 1: Dequeue 0, enqueue neighbors 1,3 $\rightarrow$ Queue:[1,3]. Step 2: Dequeue 1, enqueue unvisited neighbors 2,4 $\rightarrow$ Queue:[3,2,4]. Step 3: Dequeue 3, neighbors already visited $\rightarrow$ Queue:[2,4]. Step 4: Dequeue 2, enqueue 5 $\rightarrow$ Queue:[4,5]. Step 5: Dequeue 4 $\rightarrow$ Queue:[5]. Step 6: Dequeue 5 $\rightarrow$ Queue empty. BFS order: 0,1,3,2,4,5.

Why Queue? The queue ensures FIFO ordering — vertices discovered earlier are processed earlier. This automatically explores level 1 (distance-1 neighbors) before level 2, ensuring minimum-distance exploration.

Complexity: Time $O(V+E)$ with adjacency list. Space $O(V)$ for the queue. Applications: shortest path, web crawling, social network friend suggestions, network broadcasting, garbage collection (tri-color marking).

Deque Definition: A Double-Ended Queue (Deque) supports insertion and deletion at both ends. It generalizes both stack and queue. It can be implemented using a circular array with two pointers (front, rear) or a doubly linked list.

InsertFront(X): Check overflow: $(front-1+MAX)\%MAX = rear$. If empty: $front=rear=0$. Else: $front = (front-1+MAX)\%MAX$. Set $D[front] = X$. Example: Deque=[A,B,C], InsertFront(X) $\rightarrow$ [X,A,B,C]. Front moves backward.

InsertRear(Y): Check overflow. If empty: $front=rear=0$. Else: $rear = (rear+1)\%MAX$. Set $D[rear] = Y$. Example: [A,B,C], InsertRear(Y) $\rightarrow$ [A,B,C,Y]. Rear moves forward.

DeleteFront(): Check underflow. Save $val=D[front]$. If $front==rear$: reset to -1. Else: $front = (front+1)\%MAX$. Return val. Example: [A,B,C], DeleteFront() removes A $\rightarrow$ [B,C].

DeleteRear(): Check underflow. Save $val=D[rear]$. If $front==rear$: reset to -1. Else: $rear = (rear-1+MAX)\%MAX$. Return val. Example: [A,B,C], DeleteRear() removes C $\rightarrow$ [A,B].

All operations run in O(1) time. Applications: Sliding window maximum, palindrome checking, work-stealing parallel schedulers (Java ForkJoin), undo/redo in editors, and simulating both stack and queue with a single structure.

⚡ **One-Page Quick Revision — Unit IV: Queues & Deques** — Review this sheet before your exam!

📌 Queue — Key Definitions

- **Queue:** FIFO linear data structure
- **Front:** Deletion end
- **Rear:** Insertion end
- **Overflow:** Insert into full queue
- **Underflow:** Delete from empty queue
- **Circular Queue:** Ring-based wrap-around

🕒 Time Complexities

- Enqueue: **O(1)**
- Dequeue: **O(1)**
- Peek/Front: **O(1)**
- isEmpty: **O(1)**
- isFull: **O(1)**
- Traversal: **O(n)**
- BFS: **O(V+E)**

🟦 Circular Queue Conditions

- Empty: $\text{front} == -1$
- Full: $(\text{rear}+1)\% \text{MAX} == \text{front}$
- Enqueue: $\text{rear} = (\text{rear}+1)\% \text{MAX}$
- Dequeue: $\text{front} = (\text{front}+1)\% \text{MAX}$
- Size: $(\text{rear}-\text{front}+\text{MAX})\% \text{MAX}$

🟠 Deque — Types & Ops

- **Input-Restricted:** Insert at rear only; delete both ends
- **Output-Restricted:** Delete at front only; insert both ends
- InsertFront, InsertRear: **O(1)**
- DeleteFront, DeleteRear: **O(1)**
- Deque = Stack + Queue

🗺️ BFS — Quick Facts

- Uses a **Queue** (FIFO)
- Visits level by level
- Time: **O(V+E)** (adj. list)
- Space: **O(V)**
- Finds shortest path (unweighted)
- visited[] prevents revisiting

📄 CPU Scheduling

- FCFS = First Come First Served
- Uses a simple Queue
- $\text{WT} = \text{CT} - \text{AT} - \text{BT}$
- $\text{TAT} = \text{CT} - \text{AT}$
- Round-Robin = Circular Queue
- Non-preemptive (basic FCFS)

✅ Array vs LL Queue

- Array: Fixed size, simpler, cache-friendly
- Linked List: Dynamic, no overflow
- Both: **O(1)** enqueue/dequeue
- LL: Extra pointer memory
- Circular array fixes array issues

🎯 Applications Summary

- BFS graph traversal
- CPU scheduling (FCFS, RR)
- Print spooler / IO buffers
- Keyboard input buffer
- Sliding window (Deque)

- Palindrome check (Deque)

FIFO Concept Summary

The FIFO principle is the defining characteristic of a queue: **the element that has been in the queue the longest is always the next to be processed**. This ensures fairness (no element is bypassed) and order preservation (elements are processed in arrival order). FIFO is essential for: CPU scheduling fairness, network packet ordering, print job sequencing, and BFS level-by-level graph exploration. Whenever a problem requires processing in the same order as arrival — think Queue.

One-Line Exam Answers

Term	One-Line Definition
Queue	Linear FIFO structure with insertion at rear and deletion at front.
Enqueue	Insert element at rear: <code>Q[++rear] = x</code>
Dequeue	Remove element at front: <code>return Q[front++]</code>
Circular Queue	Queue using array where rear wraps to 0 via modulo arithmetic.
Deque	Double-ended queue allowing insert/delete at both front and rear.
BFS	Graph traversal using queue, visiting all neighbors before deeper levels.
FCFS	CPU scheduling where the first process to arrive is served first.
Input-Restricted Deque	Insertion only at rear; deletion at both ends.
Output-Restricted Deque	Deletion only at front; insertion at both ends.



Prescribed Textbooks

- [T1] Weiss, Mark Allen. *Data Structures and Algorithm Analysis in C*, 2nd Edition. Pearson Education, 1997. ISBN: 978-0201498400.
→ Recommended chapters: Ch. 3 (Lists, Stacks, Queues), Ch. 9 (Graph Algorithms including BFS)
- [T2] Horowitz, Ellis, and Sartaj Sahni. *Fundamentals of Data Structures in C*, 2nd Edition. Silicon Press, 2007. ISBN: 978-0929306407.
→ Recommended chapters: Ch. 4 (Stacks and Queues), Ch. 6 (Graphs)



Reference Books

- [R1] Mehlhorn, Kurt, and Peter Sanders. *Algorithms and Data Structures: The Basic Toolbox*. Springer, 2008. ISBN: 978-3540779773.
→ Advanced treatment of sequence data structures and amortized analysis.
- [R2] Aho, Alfred V., Jeffrey D. Ullman, and John E. Hopcroft. *Data Structures and Algorithms*. Addison-Wesley, 1983. ISBN: 978-0201000238.
→ Classical reference for formal ADT treatment of queues and graph algorithms.
- [R3] Miller, Brad, and David Ranum. *Problem Solving with Algorithms and Data Structures using Python*. Franklin, Beedle & Associates, 2011.
→ Excellent for conceptual understanding; includes interactive queue simulations.
- [R4] Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, 4th Edition. MIT Press, 2022. ISBN: 978-0262046305.
→ Recommended sections: Ch. 10 (Elementary Data Structures), Ch. 22 (BFS & DFS)
- [R5] Sedgewick, Robert. *Algorithms in C, Parts 1–4: Fundamentals, Data Structures, Sorting, Searching*, 3rd Edition. Addison-Wesley, 1997. ISBN: 978-0201314526.
→ Practical C implementations of all queue variants with performance benchmarks.
- [R6] Knuth, Donald E. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd Edition. Addison-Wesley, 1997.
→ Mathematical analysis of queue operations and historical context.

 **Module Summary:** This module covered the complete syllabus of Unit IV — Queues & Deques for B.Tech 1st Year Data Structures. Topics included: Introduction to Queues (FIFO principle, properties), Queue operations (Enqueue, Dequeue, Peek) with algorithms and diagrams, Array and Linked List implementations, Circular Queue with modular arithmetic, Applications (BFS, CPU scheduling, spoolers), Introduction to Deques with all four operations, and Applications of Deques (sliding window, palindrome, work-stealing). All diagrams are exam-ready HTML/CSS/SVG. Questions cover both 2-mark and 5-mark formats as per university examination pattern.

Data Structures — Unit IV

QUEUES & DEQUES · B.TECH 1ST YEAR · ALL RIGHTS ACADEMIC

